

Minimal Cangjie for Arbitrary Algorithms

<https://gemini.google.com/app/ae45f3f06f6663aa>

User prompt: consider the <https://cangjie-lang.cn/en/docs> cangjie programming language. What is the smallest fragment that allows arbitrary algorithmics and composition. Features that are simply for the sake of abstraction (e.g. generics, inheritance) can be left out.

Response: To achieve Turing completeness and arbitrary algorithmic composition in Cangjie while stripping away abstraction-heavy features (like class, interface, generics, macros, and concurrency), you need a very small subset of the language.

The absolute minimal fragment of Cangjie required to express any computable algorithm consists of **Functions, Primitive State, Basic Control Flow, and Algebraic Data Types**.

Here is the smallest viable subset:

1. Primitive State (let, var, Int64, Bool)

You need a basic numeric type, a boolean type for logic, and the ability to bind memory.

- **Int64** and **Bool** (The only types you strictly need for algorithmic logic).
- **let** for immutable bindings (safe, predictable state).
- **var** for mutable variables (required for iterative algorithms in place).

2. Callable Logic (func)

Functions are the fundamental unit of composition. They allow you to reuse logic and execute recursive algorithms.

- Top-level **func** definitions with typed parameters and return types.

3. Control Flow (if, while)

Because Cangjie is expression-oriented, these constructs are highly concise.

- **if/else**: For conditional branching. In Cangjie, **if** is an expression and can return a value directly.
- **while**: The fundamental looping primitive for unbounded iteration (omitting **for-in**, which is essentially syntactic sugar for iterators).

4. Data Composition (enum and match)

To process arbitrary amounts of data (acting as the memory "tape" for a Turing machine), you need a way to build recursive data structures like lists and trees. Rather than using OOP **class** references or standard library **Array<T>** (which requires generics), Cangjie's **Algebraic Data Types (enum)** provide the smallest, mathematically purest way to compose unbounded data.

- **enum**: Sum types that can hold data in their constructors (e.g., building a Linked List or Binary Tree).
- **match**: The control flow required to safely destructure and read data from enums.

The Minimal Fragment in Action

By combining just these features, you can write any arbitrary algorithm without needing classes, generics, or standard library collections. Here is how this minimal fragment looks when composing a functional linked list and an iterative algorithm:

```
// 1. DATA COMPOSITION (No classes or generics needed)
enum IntList {
  | Nil
  | Cons(Int64, IntList)
}

// 2. CALLABLE LOGIC & RECURSION
func createRange(start: Int64, end: Int64): IntList {
  if (start >= end) {
    return Nil
  } else {
    return Cons(start, createRange(start + 1, end))
  }
}

// 3. STATE & CONTROL FLOW
func sumList(list: IntList): Int64 {
  var sum: Int64 = 0 // Mutable primitive state
  var current = list
```

```

while (true) {          // Unbounded iteration
  match (current) {     // Destructuring composition
    case Nil =>
      break
    case Cons(val, next) =>
      sum = sum + val
      current = next
  }
}
return sum
}

// Execution
func main(): Unit {
  let numbers = createRange(1, 100)
  let total = sumList(numbers)
}

```

What You Safely Leave Out:

If you are strictly focused on algorithmics, you can entirely ignore:

- **Object-Oriented Abstractions:** class, interface, public/private modifiers, open, and override.
- **Type Abstractions:** Generics (`<T>`), Type Extensions (extend).
- **Metaprogramming:** Macros (quote) and operator overloading.
- **Runtime Complexities:** Concurrency (spawn), Exceptions (try/catch).

Sources

1. [Learn Cangjie in Y Minutes](#)

User prompt: write a priority queue using heaps in his fragment

Response: To implement a priority queue within this strictly minimal fragment, we cannot use a traditional array-based binary heap. Array-based heaps rely on contiguous memory and index math (e.g., $2*i + 1$), which requires standard library collections and generics.

Instead, we can use a **Skew Heap**—a self-adjusting, pointer-based binary tree that is perfectly suited for Algebraic Data Types (enum). A Skew Heap provides $O(\log n)$ amortized performance for insertions and extractions entirely through functional merging, without needing to track tree depth or maintain a strict complete binary tree shape.

Here is how you construct a Min-Priority Queue using a Skew Heap in the minimal Cangjie fragment:

1. The Algebraic Data Structure

The heap is simply a binary tree where every node guarantees that its value is smaller than or equal to its children (the heap property).

```

enum PriorityQueue {
  | Empty
  | Node(Int64, PriorityQueue, PriorityQueue) // val, left_child, right_child
}

// A custom type to return the result of an extraction,
// since we are excluding standard Tuples or Option<T>.
enum PopResult {
  | None
  | Just(Int64, PriorityQueue) // The extracted min value, and the new queue
}

```

2. The Core Operation: Merge

In a Skew Heap, both insertion and extraction are just specific cases of **merging** two heaps. To merge, we compare the roots. The smaller root becomes the new root. We then unconditionally swap its left and right children, and recursively merge the other heap with the *new* left child.

```

func merge(h1: PriorityQueue, h2: PriorityQueue): PriorityQueue {
  match (h1) {
    case Empty =>
      return h2
    case Node(v1, l1, r1) =>
      match (h2) {
        case Empty =>
          return h1
        case Node(v2, l2, r2) =>
          if (v1 <= v2) {
            // h1 is smaller: swap its children and merge h2 with the old right child
            return Node(v1, merge(r1, h2), l1)
          } else {
            // h2 is smaller: swap its children and merge h1 with the old right child
            return Node(v2, merge(r2, h1), l2)
          }
      }
  }
}

```

3. Insert and Extract-Min

With merge defined, the priority queue API becomes trivial.

Insertion is simply merging the existing heap with a single-node heap:

```

func insert(pq: PriorityQueue, val: Int64): PriorityQueue {
  let singleNode = Node(val, Empty, Empty)
  return merge(pq, singleNode)
}

```

Extraction removes the root (which is guaranteed to be the minimum) and merges its left and right branches back together:

```

func popMin(pq: PriorityQueue): PopResult {
  match (pq) {
    case Empty =>
      return None
    case Node(minVal, left, right) =>
      // The new heap is just the merge of the two subtrees
      let newQueue = merge(left, right)
      return Just(minVal, newQueue)
  }
}

```

Usage Example

This structure gives you a fully functional, arbitrary-capacity priority queue using nothing but algebraic destructuring and recursion.

```

func main(): Unit {
  var pq = Empty

  // Insert values
  pq = insert(pq, 42)
  pq = insert(pq, 10)
  pq = insert(pq, 15)

  // Extract the minimum
  match (popMin(pq)) {
    case None =>
      // Queue was empty
    case Just(val, rest) =>
      // val will be 10
      pq = rest // Update the state with the remaining queue
  }
}

```

User prompt: whats the deal with arrays?

Response: Arrays break the rules of a *minimal* algorithmic fragment because they introduce a hard dependency on hardware-level memory semantics and higher-order language abstractions.

Here is why standard arrays are left out of a minimal core:

1. **The Generics Problem:** A useful array must be polymorphic (`Array<T>`). If you strip out generics to keep the language fragment mathematically minimal, you are forced to define separate, hardcoded types for every array variant (`Int64Array`, `BoolArray`, `ListArray`), which destroys compositional elegance.
2. **Contiguous Allocation & Intrinsic:** Unlike Algebraic Data Types (ADTs) which allocate memory node-by-node, arrays require allocating a single, contiguous block of memory. In safe languages like Cangjie, exposing $O(1)$ indexing (`arr[i]`) requires hidden compiler intrinsics, pointer arithmetic, and runtime bounds-checking. It cannot be expressed purely through the language's own syntax.
3. **Mutability and Reallocation:** Arbitrary algorithmics require unbounded data (like an infinite Turing machine tape). A primitive array has a fixed size. To make it unbounded, you have to build a dynamic array (like a `Vec` or `ArrayList`), which requires object-oriented encapsulation to manage capacity, track length, and handle memory reallocation safely under the hood.

The ADT Alternative

Algebraic Data Types (enum) bypass all of this. They allow you to define unbounded, recursive data structures using purely inductive mathematical logic. The runtime handles the heap allocation node-by-node, allowing arbitrary composition without needing contiguous memory mapping, index math, or generics.

User prompt: what are the performance trade-offs

Response: The fundamental trade-off is between **algorithmic purity** and **mechanical sympathy**. The minimal ADT fragment provides perfectly clean inductive semantics, but it clashes violently with how modern hardware architectures actually execute instructions.

Spatial Locality and Cache Prefetching

Modern CPUs do not fetch memory byte-by-byte; they fetch 64-byte **cache lines**.

- **Arrays:** Because arrays map to contiguous physical memory, reading `arr[0]` automatically pulls the adjacent elements into the L1 cache. The hardware prefetcher recognizes the sequential access pattern and streams subsequent cache lines into L1 before the CPU even requests them.
- **Pointer-based ADTs:** A Skew Heap or functional Linked List allocates nodes arbitrarily across the heap. Traversing `node.left` or `node.next` requires reading a memory address, pausing execution, and fetching from that arbitrary address (pointer chasing). The CPU cannot predict the next jump, leading to constant L1 cache misses.

Key insight: A single main-memory fetch (a cache miss) takes roughly 100x longer than reading a prefetched value from L1. In practice, a theoretical $O(n)$ array traversal will consistently outrun a theoretical $O(\log n)$ tree traversal if the tree is small enough to fit inside the array's cache footprint.

Memory Footprint (Indirection Overhead)

ADTs drastically bloat the working set size, further thrashing the cache.

If you store one million 64-bit integers (`Int64`):

- **Array:** ~8 MB of flat memory.
- **Skew Heap (`Node(Int64, Heap, Heap)`):** Each node requires 8 bytes for the integer, plus two 8-byte pointers (16 bytes), plus object header overhead for enum tags and GC tracking (typically 8-16 bytes). A single integer now consumes roughly 32 to 40 bytes. The 8 MB payload expands to a ~40 MB footprint.

Allocation and Garbage Collection

In an array-backed structure (like a standard Binary Heap), expanding capacity involves infrequent, amortized block reallocations. Insertion is otherwise allocation-free.

With pure ADTs, *every* insertion requires a distinct heap allocation. Building a data structure of 10,000 items means invoking the memory allocator 10,000 times. This incurs direct CPU overhead during construction and places immense pressure on the Garbage Collector, as thousands of small, disjointed nodes must eventually be traced and swept.

Summary of Extremes

Metric	Contiguous Array	Pure ADT (Minimal Fragment)
Memory Access	Predictable, hardware-prefetched	Random, pointer chasing
Space Efficiency	Optimal (data only)	High overhead (pointers + tags)

Metric	Contiguous Array	Pure ADT (Minimal Fragment)
Allocation Cost	Amortized $O(1)$ bulk allocation	High $O(1)$ individual heap alloc per node)
Semantics	Imperative, state-dependent	Inductive, functionally pure

User prompt: how are cangjie arrays implemented?

Response: Cangjie implements arrays by splitting them into two distinct memory models to balance safety and performance: heap-allocated reference types (`Array<T>`) and stack-allocated value types (`VArray<T, N>`).

1. Reference Arrays (`Array<T>`)

The standard `Array` in Cangjie is a reference type. When you initialize one, the runtime allocates a contiguous block of memory on the heap, which is tracked and eventually cleaned up by the Garbage Collector (GC).

- **Immutability of Size:** While the individual elements inside the array can be modified, the overall length of the `Array` type is fixed at the moment of creation.
- **Semantics:** Passing an `Array` to a function passes a reference (pointer) to the heap allocation, not a copy of the data.

2. Value Arrays (`VArray<T, N>`)

To support high-performance systems programming without triggering GC overhead, Cangjie provides `VArray` (Value Array).

- **Stack Allocation:** These are pure value types. Their memory is allocated directly on the stack frame (or inlined directly inside a parent struct).
- **Compile-Time Sizing:** The length `N` must be a hardcoded constant at compile time.
- **Abstraction Limits:** Because a `VArray` is a raw memory value rather than a fully-fledged object, it cannot implement interfaces.

3. Dynamic Arrays (`ArrayList<T>`)

Because primitive arrays have fixed lengths, unbounded algorithmic logic requires the `std.collection` package.

- **Under the Hood:** An `ArrayList` is a dynamic object that wraps a standard `Array<T>`.
- **Reallocation:** When you call functions like `append` or `insert`, the list checks its internal capacity. If the backing array is full, the `ArrayList` requests a larger contiguous block from the heap, copies the old elements over, and updates its internal pointer to the new block.

Sources

- 1. [Docs-The Cangjie Programming Language](#)
- 2. [Interfaces](#)

User prompt: using the small fragment identified above can you implement varray?

Response: The short answer is: **No, you cannot implement a true `VArray` (Value Array) using this minimal fragment.**

A true stack-allocated array is an abstraction over hardware-level contiguous memory. It requires compiler intrinsics that are entirely outside the bounds of inductive functional semantics.

Here is why `VArray` structurally violates the minimal fragment, and what happens if you try to fake it.

1. The Missing Generics (`<T, N>`)

To define a true `VArray`, you need to parameterize both the *type* of the data and the *size* of the array at compile time (`VArray<Int64, 3>`). Since generics and const-generics are excluded from our fragment, we cannot write a general-purpose definition. We would have to hardcode a specific type for every size.

2. The $O(N)$ Indexing Problem

Let's attempt to "fake" a fixed-size contiguous block using the only data composition tool we have: `enum`. We can define a hardcoded tuple representing a `VArray` of exactly 3 integers.

```
enum VArray3 {
  // A single variant containing 3 contiguous integers
  | Mk(Int64, Int64, Int64)
}
```

This creates a fixed-size structure, but we hit a fatal wall when we try to *access* an element by its index (*i*). The minimal fragment has no hardware pointer arithmetic. We cannot do `BaseAddress + (i * 8 bytes)`. The only way to extract data from an enum is via `match`.

Because `match` requires static destructuring, a variable index *i* forces us to write out an $O(N)$ linear scan:

```
func getElement(arr: VArray3, index: Int64): Int64 {
  match (arr) {
    case Mk(e0, e1, e2) =>
      if (index == 0) {
        return e0
      } else if (index == 1) {
        return e1
      } else if (index == 2) {
        return e2
      } else {
        return -1 // Out of bounds
      }
  }
}
```

If you wanted to fake `VArray1000`, the `getElement` function would require 1,000 `if/else` branches. This proves that $O(1)$ array indexing is not a mathematical consequence of the language's composition rules—it is a hardware-level intrinsic injected by the compiler.

3. Stack vs. Heap Semantics

Even if we tolerate the $O(N)$ indexing, our `VArray3` enum is still not a true `VArray` because of where it lives in memory.

- **True VArray:** The compiler guarantees the memory is allocated directly inside the current function's stack frame. When the function returns, the memory is instantly reclaimed by popping the stack pointer.
- **Minimal Fragment enum:** Algebraic Data Types are universally treated as reference types or heap-allocated boxes by the runtime when they are passed around or stored. You cannot force an `enum` to have pure C-style stack semantics without bringing back `struct` (which we excluded) or relying on silent compiler escape-analysis optimizations.

Key insight: You can implement *Turing-complete logic* with the minimal fragment, but you cannot implement *mechanical sympathy*. Features like `VArray` exist precisely because the mathematically pure core is insufficient for high-performance systems programming.

User prompt: it's ok to hardcode the definition for every size.

Response: Even if we hardcode the size to avoid the generics problem (e.g., explicitly defining `VArray10`, `VArray100`), we still cannot achieve true array semantics. The fatal flaw is **mutation**.

In the minimal fragment, Algebraic Data Types (`enum`) are structurally immutable. When you pattern match against an `enum`, the extracted fields are bound as immutable `let` values. You cannot perform an in-place memory update like `arr[5] = 42`.

The $O(N)$ Write Problem

To "mutate" an element in our hardcoded `enum`, we must reconstruct and return a completely new instance, copying over all the unmodified elements.

Here is what the "write" operation looks like if we hardcode a 4-element array:

```
enum VArray4 {
  | Mk(Int64, Int64, Int64, Int64)
}

func setElement(arr: VArray4, index: Int64, newValue: Int64): VArray4 {
  match (arr) {
    case Mk(e0, e1, e2, e3) =>
      if (index == 0) {
        return Mk(newValue, e1, e2, e3)
      } else if (index == 1) {
```

```

        return Mk(e0, newValue, e2, e3)
    } else if (index == 2) {
        return Mk(e0, e1, newValue, e3)
    } else if (index == 3) {
        return Mk(e0, e1, e2, newValue)
    } else {
        return arr // Out of bounds, return unchanged
    }
}
}

```

If you hardcode `VArray1000`, changing a single value requires:

1. Executing up to 1,000 `if` statements (an $O(N)$ linear scan).
2. Pushing 1,000 arguments to the stack to call the `Mk` constructor.
3. Allocating a brand new `VArray1000` object in memory, copying 999 old values and 1 new value (an $O(N)$ memory copy).

Why Structs Solve This

This is exactly why high-performance languages introduce `struct` as a distinct concept from `enum`.

If we step slightly outside our minimal fragment and allow `struct`, we gain **in-place mutability** and **stack allocation**:

```

struct VArray4 {
    var e0: Int64
    var e1: Int64
    var e2: Int64
    var e3: Int64
}

```

Because struct fields can be declared with `var`, we can modify memory in place (`myStruct.e2 = 42`) without copying the entire payload. However, even with structs, we still can't index them dynamically with `[i]` without compiler intrinsics translating that index into a byte offset.

Key insight: You can simulate the *logic* of an array by hardcoding an `enum` for every size, but you end up with an immutable tuple that requires $O(N)$ time and $O(N)$ memory churn for every read and write operation.

User prompt: when i asked you to identify a minimal algorithmic fragment i didn't mean just expressiveness but also performance: any program written in the full cangjie must be able to be written in the small cangjie without loss of time or space performance. with this in mind, what is the minimal fragment?

Response: If performance parity (both time and space) is a strict requirement, the definition of "minimal" shifts fundamentally. We can no longer rely on purely functional Algebraic Data Types (`enum`) because they inherently mandate heap allocation, pointer indirection, and branch-heavy $O(N)$ lookups.

To guarantee that any program in the full language can run with identical $O(1)$ time complexity and exact byte-for-byte memory density in the restricted subset, the minimal fragment must directly expose Cangjie's hardware-sympathetic memory layout. Essentially, the fragment becomes Cangjie translated to a safe subset of C.

Here is the strict performance-parity minimal fragment:

1. Unboxed Aggregation (`struct`)

To achieve zero-overhead allocation and dense memory packing, `struct` must replace `enum` as the primary data composition tool.

- **Stack Allocation:** Structs allow you to aggregate data entirely on the stack (or inlined within other allocations), entirely bypassing the garbage collector.
- **In-Place Mutability:** Defining fields with `var` inside a struct allows true $O(1)$ in-place memory updates without the $O(N)$ copying penalty of immutable ADTs.

2. Intrinsic Contiguous Memory (`CPointer<T>`)

You must retain the hardware indexing intrinsic. The mathematically minimal way to do this in Cangjie without relying on generic standard library types like `Array<T>` is via the unsafe pointer primitive.

- **`CPointer<T>`:** Provides raw access to contiguous memory blocks.

- **Pointer Arithmetic / Indexing:** Allows the compiler to emit direct load/store instructions ($\text{Base Address} + (\text{Index} * \text{Element Size})$). This guarantees $O(1)$ spatial locality and triggers hardware cache prefetching perfectly.

3. Primitive State & Control Flow (Int64, Float64, Bool, func, if, while)

- **Primitives:** You need the actual unboxed hardware types to perform ALU operations without boxing overhead.
- **while:** The fundamental looping construct. Iterators (`for-in`) can be manually lowered to `while` loops over pointers to achieve identical zero-overhead iteration.
- **func and if:** For standard branching and callable logic.

4. Sealed GC Objects (Final class)

If a program relies on Cangjie's concurrent Garbage Collector for specific throughput profiles (rather than manually managing memory with `CPointer`), you need the ability to allocate traced heap objects.

- **Final class:** You only need a standard class *without* inheritance modifiers (`open`, `override`). This provides a GC-managed heap allocation with standard object headers, yielding the exact same allocation footprint as the full language.

Why the Rest is Excluded (How to manually lower them)

To prove this fragment is sufficient for peak performance, here is how the compiler (or you, manually) can lower all abstraction-heavy features into this fragment with **zero runtime cost**:

- **Generics (<T>):** Under Cangjie's operational semantics, generics are reified. You can achieve identical performance simply through manual **monomorphization**. If the full program uses `Array<Int64>` and `Array<Float64>`, you manually write `Int64Array_struct` and `Float64Array_struct`. The source code balloons, but the compiled LLVM IR and execution time are byte-for-byte identical.
- **Inheritance and Interfaces (interface):** Dynamic dispatch requires a v-table (virtual method table). You can manually lower this by creating a `struct` that contains function pointers (`CPointer<() -> Unit>`). Calling a function pointer through a `struct` is precisely what the hardware does when it resolves an `override` method.
- **Pattern Matching (match):** Can be lowered to a `switch` or `if/else` block reading an integer tag on a `struct` (simulating a C-style tagged union), resulting in the exact same branch-prediction performance as native matching.

Key insight: When space and time performance are non-negotiable constraints, a language's minimal fragment reduces directly to its lowest-level memory abstractions. Everything else—generics, traits, objects, and pattern matching—is just syntactic sugar that the compiler lowers into `structs`, `pointers`, and `function calls`.

User prompt: is this language fragment fully axiomatizable in a hoare logic?

Response: Not in *classical* Hoare logic. By introducing `CPointer<T>` and mutable heap-allocated `class` references to achieve zero-overhead performance parity, the fragment introduces **unrestricted aliasing**.

Classical Hoare logic structurally fails in the presence of memory aliasing. To fully axiomatize this fragment, you must extend the calculus with **Separation Logic**.

Here is a breakdown of why the classical calculus fails on this specific fragment, and the spatial axioms required to fix it.

The Breakdown of the Assignment Axiom

The foundational rule of classical Hoare logic is the assignment axiom, which relies entirely on syntactic substitution:

$$\{P[E/x]\} x := E \{P\}$$

This assumes that variable names map cleanly to independent memory locations. If your fragment uses a `CPointer` (or two `class` variables pointing to the same object) to mutate memory, this axiom collapses.

If pointer x and pointer y are aliases for the same memory address, and we assert that y points to 10: $\{ *y = 10 \} *x := 42$
 $\{ *y = 10 \text{ (Invalid!)} \}$

Because classical Hoare logic has no built-in model of the heap, a mutation to x cannot logically invalidate the state of y through pure substitution.

Axiomatizing the Fragment via Separation Logic

To construct a sound axiomatic semantics for this performance-oriented fragment, you must introduce the **points-to predicate** ($x \mapsto v$) and the **separating conjunction** ($\star$).

This shifts the logic from reasoning purely about the variable environment (the stack) to reasoning spatially about disjoint regions of the heap.

1. Pointer and Class Mutation

Instead of the classical assignment axiom, updating a `CPointer` or a `class` field requires spatial ownership. The mutation axiom states that you can only update memory if you can prove you "own" that specific footprint: $\{E \mapsto -\} * E := F \{E \mapsto F\}$

2. Local State and Stack Structs

Stack-allocated `struct` values and primitive `Int64` loop counters do not inherently alias. They can be modeled efficiently by mapping struct fields directly to the local variable environment rather than the spatial heap, allowing you to retain the simpler classical substitution axiom for the non-pointer parts of the algorithmic loop.

3. Algorithmic Composition (The Frame Rule)

To achieve the "arbitrary composition" you originally asked for, algorithms verified in isolation must be composable without re-verification. In classical Hoare logic, a function call might secretly mutate a global pointer.

Separation logic enables composition via the **Frame Rule**:

$$\frac{\{P\} C \{Q\}}{\{P \star R\} C \{Q \star R\}}$$

This rule formally guarantees that if an algorithmic command C executes safely in memory footprint P , it will execute safely in a larger heap state $(P \star R)$ without modifying the disjoint frame R . This is the exact mathematical foundation used by static analysis frameworks like Infer to scale compositional verification across massive codebases involving pointers and objects.

The Garbage Collection Caveat

Because the fragment includes `class`, the operational semantics of the language include a Garbage Collector. While Separation Logic perfectly axiomatizes the *functional correctness* and *memory safety* (no null pointer dereferences) of the fragment, standard Hoare/Separation logic axioms assume explicit `free()` commands. To fully formalize the fragment's memory lifecycle, the axiomatic semantics must be combined with a reachability model or **fractional permissions** to prove that memory is safely abandoned to the GC rather than manually deallocated.

User prompt: Based on existing research is this fragment formalisable in a proof assistant? Which proof assistant would you recommend?

Response: Yes, this fragment is entirely formalizable. Because our minimal Cangjie fragment behaves like a safe subset of C (with `CPointer` and stack-allocated `structs`) combined with a garbage-collected heap (`class`), we can rely on extensive existing research in formalizing imperative memory models.

To prove properties about programs written in this fragment, you need a proof assistant that has strong, pre-existing libraries for **Separation Logic**.

For this specific task, **Coq** is the most highly recommended proof assistant. It hosts two major frameworks perfectly suited for this:

- **The Verified Software Toolchain (VST)**: Built explicitly to verify C programs, it already has the mathematical machinery to handle raw pointer arithmetic, contiguous memory, and stack frames.
- **Iris**: A state-of-the-art framework for higher-order concurrent separation logic embedded in Coq. It is heavily used to formalize languages like Rust (e.g., the RustBelt project) and can model the combination of manual pointers and GC-managed objects.

While Coq is the industry standard for this type of memory formalization, **Lean 4** and **Isabelle/HOL** are also highly capable alternatives.

To figure out how we might approach this formalization, which area would you like to explore first?

1. **Frameworks in Coq** 🐦 : How VST or Iris actually manages pointer aliasing and memory safety proofs.
2. **Lean 4** 🧱 : Why this newer proof assistant is gaining popularity and how its approach to mutable state differs from Coq.
3. **Modeling the Memory** 🧠 : How we mathematically define the difference between a `CPointer` address and a `class` reference before writing a single proof.

User prompt: suggest a good name for this fragment of cangjie

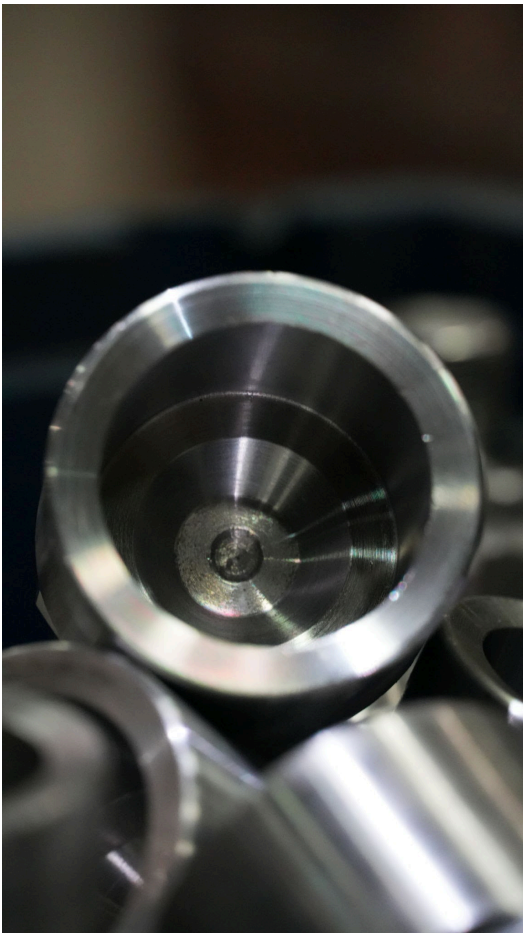
Response: Because this fragment balances high-level algorithmic logic with low-level mechanical sympathy, its name should reflect either its mathematical irreducibility or its proximity to the hardware.

Here are four ways to brand it, depending on how you intend to use it:

- **Cangjie Radical (CJ-R):** In Chinese linguistics, a "radical" is the foundational, irreducible component of a character. Since the mythical figure Cangjie invented characters, calling this the "Radical" mathematically implies it is the core building block from which all other abstractions (generics, interfaces) are constructed.



- **Cangjie BareMetal (CJ-BM):** If your focus is entirely on performance parity, this name signals the explicit inclusion of CPointer, unboxed structs, and raw memory layout, stripping away the runtime overhead to touch the hardware.



- **Axiomatic Cangjie (AxCJ):** If your goal is strictly formal verification in Coq or Lean 4, this highlights that the fragment was intentionally designed to be bound by Separation Logic and mathematical proofs rather than compiler magic.
- **Cangjie-Zero:** A nod to the "zero-overhead" philosophy of systems programming, emphasizing that any algorithmic abstraction written in this subset incurs exactly zero hidden allocations or pointer-chasing penalties.

If you had to write a paper or documentation on this fragment today, which of those angles—linguistics, hardware, or formal logic—feels truest to your goal?

